

5_mergesort

```
#include<iostream> // For input/output (cout)
#include<omp.h> // For OpenMP parallel programming
using namespace std;
// Function to merge two sorted parts of the array
void merge(int arr[], int l, int m, int r)
{
    int i, j, k; // i → left array, j → right array, k → main array index

    int n1 = m - l + 1; // Size of left subarray
    int n2 = r - m; // Size of right subarray
    int L[n1], R[n2]; // Temporary arrays to store left and right parts
    // Copy elements from original array to left subarray
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    // Copy elements from original array to right subarray
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    i = 0; // Start index for left array
    j = 0; // Start index for right array
    k = l; // Start index for merged array
    // Compare elements from both arrays and store smaller one
    while (i < n1 && j < n2)
    {
        if (L[i] <= R[j]) // If left element is smaller or equal
        {
            arr[k] = L[i]; // Put left element in main array
            i++; // Move left index
        }
        else // If right element is smaller
        {
            arr[k] = R[j]; // Put right element in main array
            j++; // Move right index
        }
        k++; // Move main array index
    }
    // Copy remaining elements of left array (if any)
    while (i < n1)
    {
        arr[k] = L[i];

        i++;
        k++;
    }
    // Copy remaining elements of right array (if any)
    while (j < n2)
    {
```

```

arr[k] = R[j];
j++;
k++;
}
}
// Recursive function for merge sort
void mergeSort(int arr[], int l, int r)
{
    if (l < r) // Continue only if more than one element
    {
        int m = l + (r - l) / 2; // Find middle index (safe way)
        // OpenMP: create parallel sections
        #pragma omp parallel sections
        {
            // First section → sort left half
            #pragma omp section
            {
                mergeSort(arr, l, m);
            }
            // Second section → sort right half
            #pragma omp section
            {
                mergeSort(arr, m + 1, r);
            }
        }
        // Merge the two sorted halves
        merge(arr, l, m, r);
    }
}

// Function to print array elements
void printArray(int A[], int size)
{
    int i;
    // Loop to print each element
    for (i = 0; i < size; i++)
        cout << A[i] << " ";
    cout << "\n"; // Move to next line
}

// Main function
int main()
{
    int n = 199; // Number of elements
    clock_t t1, t2; // Variables to measure time
    cout << "total number of elements: " << n << endl;
    int a[n]; // Declare array
    cout << "Storing elements in descending order...\n\n";
    // Fill array in descending order (worst case)

```

```
for (int i = 0; i < n; i++)
{
    a[i] = n - i; // Example: 199, 198, 197, ...
}
cout << "Actual list:\n";
printArray(a, n); // Print original array
t1 = clock(); // Start time

mergeSort(a, 0, n - 1); // Call merge sort
t2 = clock(); // End time
cout << "Sorted list:(Merge Sort)\n";
printArray(a, n); // Print sorted array
// Print execution time
printf("\nExecution Time: %f", (double)(t2 - t1) /
CLOCKS_PER_SEC);
return 0; // End program
}
```